

Sprawozdanie z projektu

Krzysztof Emerling, Jakub Morawiec, Jacek Sokół, Szymon Duda

1. Cel projektu

Celem projektu było stworzenie systemu komputerowego do rozpoznawania pozycji szachowej na podstawie obrazu z wykorzystaniem bibliotek OpenCV, TensorFlow/Keras oraz metod uczenia maszynowego. System realizuje następujący pipeline przetwarzania:

- wykrycie szachownicy,
- korekcję perspektywy,
- podział planszy na 64 pola, wraz z detekcją poprawności podziału,
- klasyfikację zawartości pól,
- rekonstrukcję pozycji szachowej do zapisu FEN.

2. Dataset

W projekcie został wykorzystany dataset ChessReD2 zawierający 10 800 zdjęć szachownic z ułożonymi pozycjami szachowymi. Zdjęcia zostały wykonane pod wieloma kątami, przy zachowaniu szachownicy w centrum kadru. Dataset został opublikowany na licencji Creative Commons Attribution-NonCommercial-ShareAlike 4.0 i nie jest dystrybuowany razem z repozytorium projektu.

Licencja: <https://creativecommons.org/licenses/by-nc-sa/4.0/>

3. Zakres prac

Prace nad projektem zostały podzielone na kilka kategorii.
Prace na datasetach:

- Anotacja części zbioru danych w celu utworzenia zbioru testowego do oceny jakości funkcji wykrywania szachownicy oraz przygotowania danych dla modelu klasyfikującego poprawność wykrycia szachownicy.
- Utworzenie zbioru danych do detekcji figur na poszczególnych polach szachownicy na podstawie oryginalnego datasetu przetworzonego pipeline'em wykrywania szachownicy.

Prace związane z OpenCV:

- Implementacja funkcji wykrywania krawędzi i odnajdywania konturu szachownicy wraz z transformacją perspektywy.
- Implementacja programu umożliwiającego skuteczną anotację obrazów z datasetu.
- Przygotowanie kompletnego pipeline'u przetwarzania obrazu do ekstrakcji pozycji szachowej i konwersji do formatu FEN.

Prace związane z uczeniem maszynowym:

- Implementacja klasyfikatora oceniającego poprawność wykrycia szachownicy z wykorzystaniem sieci CNN.
- (*) Implementacja oraz testy klasyfikatora opartego na analizie histogramów i algorytmie Random Forest do klasyfikacji figur na polach szachownicy.
- (*) Implementacja oraz testy klasyfikatora opartego na architekturze CNN do klasyfikacji figur na polach szachownicy.
- (*) Implementacja oraz testy klasyfikatora opartego na architekturze MLP do klasyfikacji figur na polach szachownicy.
- (*) Spośród trzech klasyfikatorów zostanie wybrany model zapewniający najlepszy kompromis pomiędzy szybkością działania a dokładnością klasyfikacji.

4. Podział prac i wykorzystane narzędzia

Do zarządzania projektem zostało utworzone repozytorium na GitHubie po linkiem:

<https://github.com/KrzysztofEmerling/ChessPiecesCV.git>

W projekcie wykorzystywane są następujące technologie:

- OpenCV,
- TensorFlow z nakładką Keras,
- Pygame do implementacji programu anotującego.

Do organizacji oraz podziału zadań wykorzystano tablicę projektu GitHub Projects zawierającą harmonogram realizacji poszczególnych etapów prac:

<https://github.com/users/KrzysztofEmerling/projects/7>

Krzysztof Emerling sporządził zarys projektu i nadzorował przebiegiem powstawania systemu. Przygotował on repozytorium na githubie, wspólnie z Szymonem Dudą dokonał adnotacji części zbioru obrazów szachownic, przyczynił się do wyboru i treningu modelu do klasyfikacji figur szachowych ze zdjęcia pola, a także utworzył pipeline do wyciągania obrazów pól szachownicy ze zdjęć.

Jakub Morawiec zajmował się pre-procesingiem danych ze zbioru, a także napisał funkcję wykrywającą szachownicę. Przyczynił się do wyboru i treningu modelu do klasyfikacji figur szachowych ze zdjęcia pola.

Jacek Sokół napisał funkcję służącą do generacji danych z pomocą modułu pygame oraz wykonał testy dokładności wykrywania szachownicy. Dodatkowo przyczynił się do wyboru i treningu modelu do klasyfikacji figur szachowych ze zdjęcia pola.

Szymon Duda dokonał adnotacji części zbioru obrazów szachownic, pracował nad modelem dyskryminatora, utworzył klasyfikator porównujący prawidłowe i błędne dane oraz sporządził dokumentację projektu.

5. Przygotowanie do uruchomienia

Do uruchomienia kodu projektu należy najpierw pobrać repozytorium.

```
git clone https://github.com/KrzysztofEmerling/ChessPiecesCV.git  
  
cd ChessPiecesCV
```

Następnie należy stworzyć środowisko wirtualne venv i pobrać wymagania z pliku requirements.txt

```
python3 -m venv venv  
  
source venv/bin/activate  
  
pip install -r requirements.txt
```

6. Uruchamianie kodu

W repozytorium znajdują się pliki typu interaktywnego notatnika python (ipynb), pliki pythonowe (py) oraz skrypty shellowe (sh).

W pierwszej kolejności należy uruchomić skrypt [setup_dataset.sh](#) znajdujący się w głównym folderze repozytorium, aby pobrać zbiór danych.

Po otrzymaniu rozpakowanego zbioru danych trzeba następnie wygenerować odseparowane pola szachownicy. Wykonuje się to za pomocą programu `generate_squares.py` znajdującego się w katalogu `dataset_generation` zgodnie z instrukcją zawartą w tymże folderze.

Posiadając odseparowane pola można następnie przejść do trenowania modeli. W katalogu `piece_recognition` istnieje plik `prepareDataset.ipynb` generujący faktyczny zbiór danych, który trzeba uruchomić. W następnej kolejności uruchomić należy pliki `CNN.ipynb`, `MLP.ipynb` oraz `histRF.ipynb`, w których trenują się modele.

Na koniec, plik `position_detection_pipeline.ipynb` pokazuje efekty działania modelu na danych i zostaje wykryta w nim szachownica po uruchomieniu.

Istnieje także możliwość porównania adnotacji wykonanych przez człowieka z tymi dokonanymi przez maszynę w programie `auto_manual_comparator.ipynb` (po warunkiem, że oba zbiory zostały wygenerowane).

7. Wyniki

- Kod prawidłowo realizuje korekcję perspektywy wraz z wykrywaniem szachownicy, w przypadku braku wykrycia szachownicy program zwróci błąd.
- W repozytorium porównano 3 różne modele wykrywania typów figur szachowych LSB + RandomForest, MPL, CNN, jednakże z uwagi na niezbalansowanie nie udało uzyskać się wystarczającej dokładności w żadnym z zastosowanych modeli. Na niepowodzenie części ML wpłynęły brak uwzględnienia rotacji planszy powyżej 30 deg (odwracającej szachownicę). Wpłynęło to na niepoprawne zaetykietowanie części danych treningowych. Skutkiem czego dokładność modeli wynosi około 70% gdzie modele MPL i CNN działały nieznacznie lepiej niż prostszy korzystający z RandomForest. Niska dokładność modeli nie pozwoliła na prawidłową rekonstrukcji pozycji szachowej.
- W repozytorium dokonano porównania obrazów oznaczonych ręcznie z modelami oznaczonymi przez utworzoną funkcję, uzyskując 10% pokrycia.
- Na końcu połączono metodę korekcji perspektywy wraz z wykrywaniem pól szachownicy z modelem MPL, zapisującą szachownicę do formatu FEN. Prezentującą wyniki połączenia wszystkich elementów projektu.

Rysunek 1 - Dokładność i strata modelu podczas treningu

